SenEvent

Accueil Nouvel evenement atou@… Se deconnecter

DAWM · BLOC PARTAGE 1/1 · M1 RETEL — DMI/FST/UCAD

Lab 7 — Workspace npm + packages/shared

Rapport de réalisation : un refactoring sans nouvelle fonctionnalité. Le projet devient un monorepo npm workspaces. Toute la logique Supabase — client, requêtes, formes de données — quitte les composants React pour `packages/shared`, écrite une seule fois en TypeScript. Le web l'importe via `@senevent/shared` ; le mobile en fera autant. Le test de réussite : l'app se comporte exactement comme avant.

tag : v0.7-shared | npm workspaces | TypeScript | initSupabase / getSupabase | 0 appel Supabase en dur | Build ✓ · tsc ✓ · 0 régression

01 Le problème : une logique, bientôt deux clients

L'app web est finie et marche. Mais chaque requête Supabase est écrite en dur dans un composant : `lib/supabase.js` pour le client, `.from().select()` dans `App.jsx`, `.insert()` dans `NouvelEvenement.jsx` … Aux Labs 8–10, le mobile aura besoin d'exactement la même logique. La laisser ici, c'est se condamner à la réécrire — deux copies qui divergeront. Le monorepo résout ça : un seul dépôt, un dossier partagé, importé par les deux apps.

AVANT — TOUT DANS APPS/WEB

```
senevent/
  apps/
    web/           # tout le projet, logique Supabase incluse
      src/
        lib/supabase.js        <- client + cles en dur
        App.jsx                <- .from("evenements").select(...)
        pages/NouvelEvenement  <- .from(...).insert(...)
        components/Bouton...   <- .from("inscriptions")...
  supabase/
```

APRÈS — LOGIQUE EXTRAITE DANS PACKAGES/SHARED

```
senevent/
  package.json               # racine : "workspaces"
  apps/
    web/                     # React + JS, consomme le package
      src/App.jsx            <- getEvenements()
      src/main.jsx           <- initSupabase(...)
  packages/
    shared/                  # TypeScript, agnostique de plateforme
      src/
        index.ts             # facade : re-exporte tout
        types.ts             # Evenement, Categorie, Profile
        supabaseClient.ts    # initSupabase / getSupabase
        evenements.ts        # getEvenements, creerEvenement...
        auth.ts / inscriptions.ts
      package.json           # @senevent/shared
      tsconfig.json
```

02 La racine qui déclare les workspaces

Un `package.json` à la racine transforme le dépôt en monorepo. `"workspaces"` dit à npm où chercher les sous-projets ; `"private": true` empêche une publication accidentelle de la racine. Un seul `npm install` installe tout et crée un lien symbolique de `@senevent/shared` dans les `node_modules` — le web importe alors le package local, sans passer par le registre public.

PACKAGE.JSON (RACINE)

```
{
  "name": "senevent",
  "private": true,
  "version": "1.0.0",
  "workspaces": ["apps/*", "packages/*"]
}
```

PACKAGES/SHARED/PACKAGE.JSON

```
{
  "name": "@senevent/shared",
  "version": "1.0.0",
  "type": "module",
  "main": "src/index.ts",
  "types": "src/index.ts",
  "dependencies": {
    "@supabase/supabase-js": "^2.0.0"
  }
}
```

Le scope `@senevent/` est un espace de nommage : il signale un paquet interne et évite les collisions. `"main"` pointe vers `src/index.ts`, la façade que les apps obtiennent en important le nom.

03 TypeScript, là où il apporte le plus

Six labs en JavaScript pur, volontairement. Maintenant que React est acquis, TypeScript entre au bon endroit : le code partagé, qui décrit des données. Un événement a un titre (texte), un prix (nombre), une catégorie (parmi 5 valeurs). On déclare ces formes une fois, et l'éditeur attrape `ev.titr`, un texte passé pour un nombre, ou un champ obligatoire oublié — avant l'exécution.

PACKAGES/SHARED/SRC/TYPES.TS

```
export type Categorie =
  | "concert" | "expo" | "conference" | "atelier" | "soutenance";

export interface Evenement {
  id: number;
  titre: string;
  categorie: Categorie;         // pas n'importe quel texte : 5 valeurs
  lieu_nom: string;
  date_debut: string;           // ISO 8601
  prix: number;
  image_url: string | null;     // le | null force a gerer "pas d'image"
  organisateur_id: string | null;
  created_at: string;
}

export interface NouvelEvenementInput {
  titre: string; categorie: Categorie; lieu_nom: string;
  date_debut: string; prix: number;
  image_url?: string | null;    // optionnel a la creation
  organisateur_id: string;      // obligatoire : RLS l'exige
}
```

Le `| null` n'est pas décoratif

Sur `image_url` et `organisateur_id`, il force le code à gérer le cas « pas d'image » ou « pas d'organisateur » — précisément les 5 événements seed, dont `profiles` vaut `null`. Sans ce marqueur, on oublierait ce cas et l'interface planterait sur un `undefined`. Le mode `strict` du `tsconfig.json` rend cette vérification obligatoire.

04 Le client découplé : pourquoi `initSupabase`

Le package ne peut pas lire `import.meta.env.VITE_...` : cette syntaxe est propre à Vite, et Expo lit ses variables autrement. La solution : le package ne lit rien lui-même. Chaque app lui passe l'URL et la clé au démarrage via `initSupabase(url, key)`. Le package reste agnostique — c'est ce découplage qui fera marcher le même code côté web et côté mobile, sans modification.

PACKAGES/SHARED/SRC/SUPABASECLIENT.TS

```
let client: SupabaseClient | null = null;

// L'application passe ses cles au demarrage. Le package ne lit
// JAMAIS import.meta.env : ni Vite, ni Expo, agnostique.
export function initSupabase(url: string, anonKey: string) {
  if (!client) client = createClient(url, anonKey);
  return client;
}

export function getSupabase(): SupabaseClient {
  if (!client) throw new Error("Supabase non initialise.");
  return client;
}
```

APPS/WEB/SRC/MAIN.JSX

```
import { initSupabase } from "@senevent/shared";

// C'est ici, et SEULEMENT ici, qu'on lit import.meta.env.VITE_...
initSupabase(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
);

createRoot(document.getElementById("root")).render(
  <StrictMode><App /></StrictMode>
);
```

Résultat : `main.jsx` est le seul fichier de tout le projet qui lit les variables d'environnement. Partout ailleurs — composants comme package — on ne connaît que `getSupabase()`.

05 Un contrat d'erreur unifié : `throw` plutôt que retourner

Au Lab 6, chaque appel renvoyait `{ data, error }` et le composant testait `error` ligne après ligne. Les fonctions partagées lèvent l'erreur : le composant appelant veut juste la donnée. On centralise la décision dans le package, et les deux apps gèrent tout de la même façon avec un `try / catch`.

AVANT — { DATA, ERROR } DANS LE COMPOSANT

```
// Lab 6 — dans le composant : verifier error a chaque appel
const { data, error } = await supabase
  .from("evenements")
  .select("*, profiles(nom)")
  .order("date_debut", { ascending: true });
if (error) {
  setErreur(error.message);
} else {
  setEvenements(data);
}
```

APRÈS — LA FONCTION PARTAGÉE LÈVE

```
// Lab 7 — la fonction partagee leve, le composant fait try/catch
export async function getEvenements(): Promise<Evenement[]> {
  const { data, error } = await getSupabase()
    .from("evenements")
    .select("*, profiles(nom)")
    .order("date_debut", { ascending: true });
  if (error) throw error;          // decision centralisee
  return (data ?? []) as Evenement[];
}
```

06 Brancher apps/web : des appels aux imports

Composant par composant, on remplace les appels Supabase en dur par un import du package. Le composant ne sait plus *comment* on récupère les événements — juste qu'il appelle une fonction qui les retourne.

AVANT — APP.JSX (LAB 6)

```
import { supabase } from "./lib/supabase";

const charger = async () => {
  setChargement(true);
  setErreur(null);
  const { data, error } = await supabase
    .from("evenements")
    .select("*, profiles(nom)")
    .order("date_debut", { ascending: true });
  if (error) setErreur(error.message);
  else setEvenements(data);
  setChargement(false);
};
```

APRÈS — APP.JSX (LAB 7)

```
import { getEvenements } from "@senevent/shared";

const charger = async () => {
  setChargement(true);
  setErreur(null);
  try {
    const data = await getEvenements();   // toute la requete est dans le package
    setEvenements(data);
  } catch (e) {
    setErreur(e.message);
  } finally {
    setChargement(false);
  }
};
```

FICHIER	APPEL EN DUR (AVANT)	FONCTION PARTAGÉE (APRÈS)
App.jsx	supabase.from('evenements').select(...)	**getEvenements()**
NouvelEvenement.jsx	supabase.from('evenements').insert(...)	**creerEvenement(input)**
Detail.jsx	supabase.from('evenements').delete().eq(...)	**supprimerEvenement(id)**
BoutonInscription.jsx	select / insert / delete inscriptions	**estInscrit / inscrire / desinscrire**
Auth.jsx	auth.signUp / signInWithPassword	**sInscrire(...) / seConnecter(...)**
NavBar.jsx	auth.signOut()	**seDeconnecter()**

Une fois tous les composants basculés, `lib/supabase.js` n'est plus référencé nulle part : il est supprimé. Un dernier détail, hors énoncé : `sInscrire` reçoit un 3e paramètre optionnel `nom`, pour continuer d'alimenter le trigger `on_auth_user_created` du Lab 5 ; et l'upload Storage de l'affiche reste inline via `getSupabase().storage`, seul l'`insert` passe par `creerEvenement()`.

07 Ce que la validation a réellement donné

Un lab de refactoring se juge sur deux critères : le code est bien réorganisé, et rien n'a cassé. Chaque ligne ci-dessous est une commande réelle et son résultat.

CONTRÔLE	COMMANDE / MÉTHODE	RÉSULTAT	OK
Package typé compile	npx tsc --noEmit dans packages/shared	Exit 0 — aucune erreur de type	✓
Build web	npx vite build sur apps/web	101 modules transformés, 0 erreur	✓
Extraction complète	grep -rc "supabase.from" apps/web/src	0 appel restant	✓
Consommation du package	grep -rl "@senevent/shared" apps/web/src	7 fichiers importent le package	✓
Ancien client supprimé	apps/web/src/lib/supabase.js	Fichier retiré, aucun import ne le référence	✓
Donnée toujours réelle	Réseau observé pendant le parcours	14 requêtes vers supabase.co, 0 mock JSON	✓

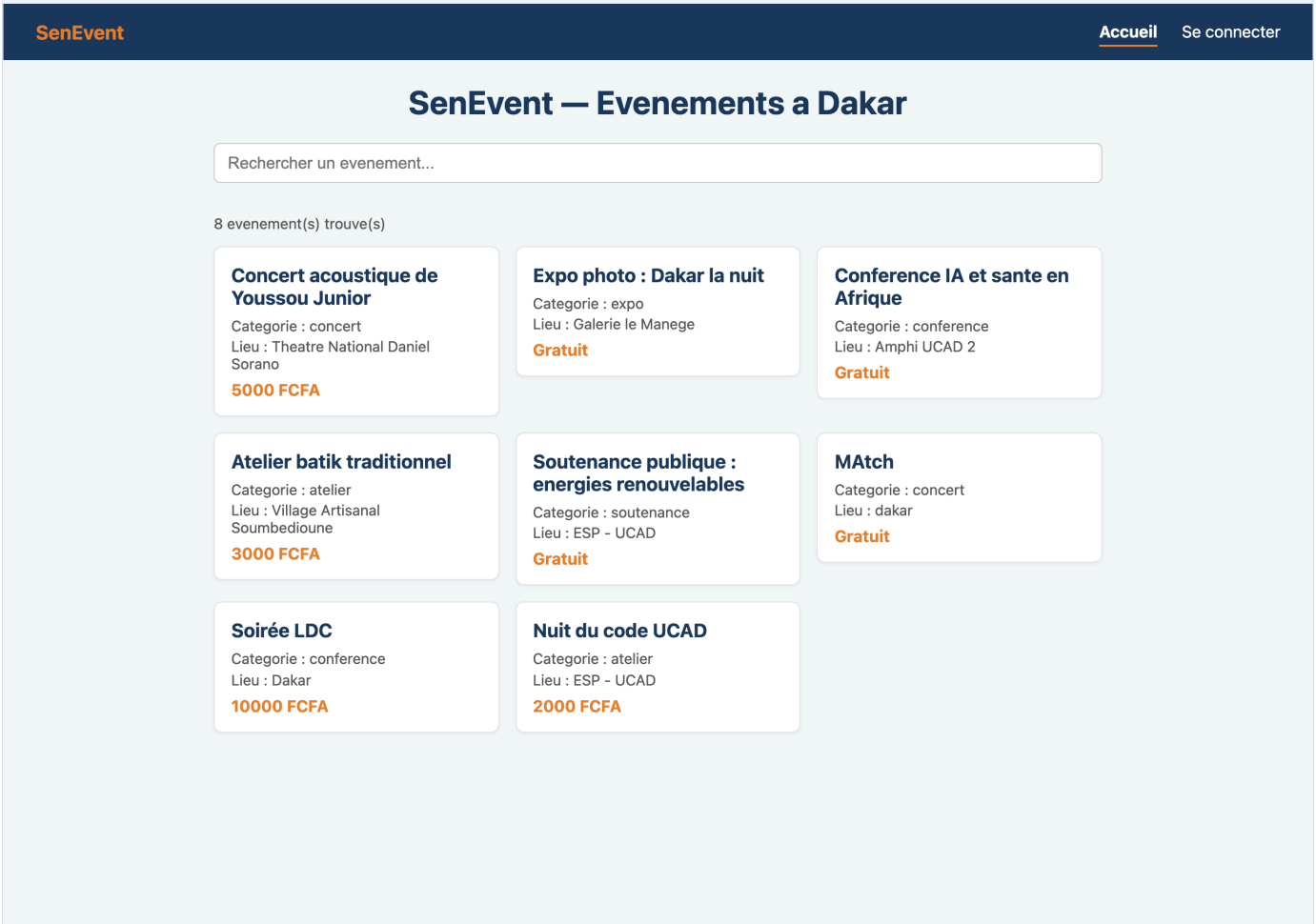
Les quatre commits suivent le découpage imposé — structure, puis logique partagée, puis branchement — et le tag `v0.7-shared` est posé sur `main` après merge : `structure monorepo` → `logique partagee` → `apps/web consomme @senevent/shared` → `Merge lab7`.

08 Non-régression, en situation

Captures prises sur l'application réelle, avec une session réelle et la vraie base. L'écran n'a pas changé depuis le Lab 6 — c'est le but. Ce qui a changé est invisible : sous chaque bouton, la logique passe maintenant par `@senevent/shared`.

FIGURE 1 /

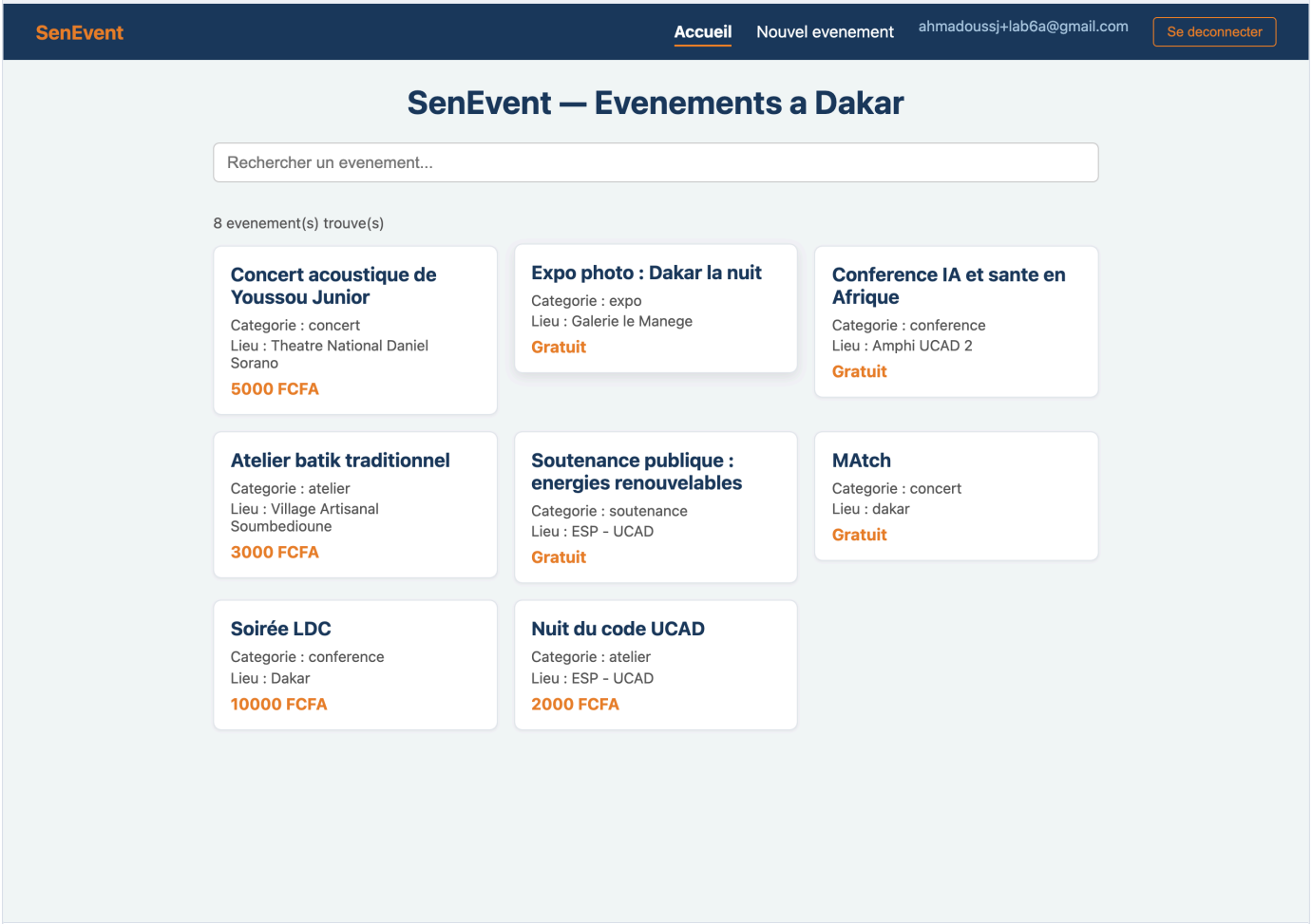
Le catalogue, servi par getEvenements()



Visuellement identique au Lab 6. Mais la liste ne vient plus d'un `.from()` écrit dans `App.jsx` : le composant appelle `getEvenements()` du package partagé. Le réseau confirme la requête exacte `select=*,profiles(nom)&order=date_debut.asc` vers `supabase.co` — la logique a déménagé, le comportement n'a pas bougé.

FIGURE 2 `/auth`

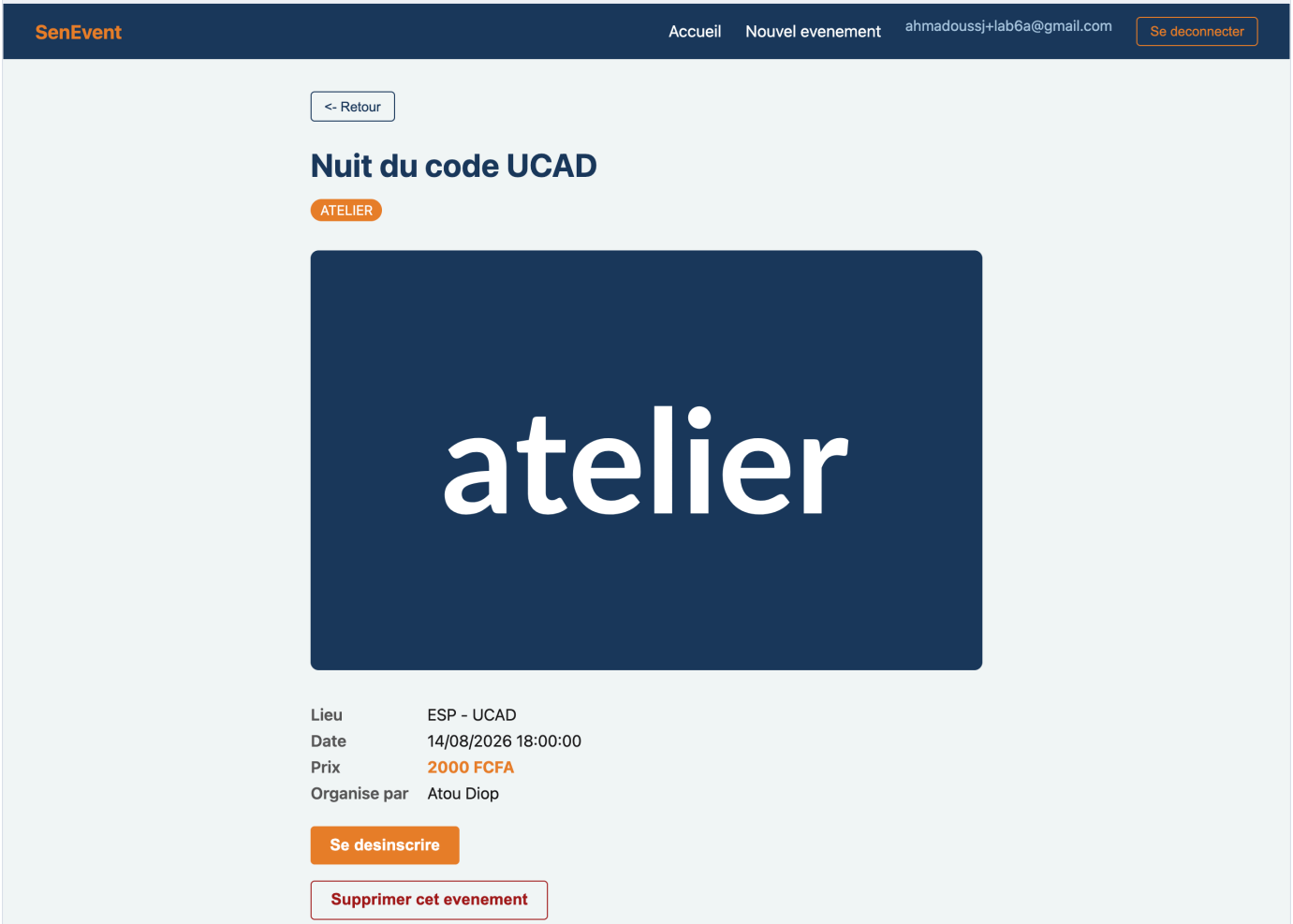
Session réelle via seConnecter()



Connexion du compte A. `Auth.jsx` n'appelle plus `supabase.auth.signInWithPassword` : il passe par `seConnecter(email, mdp)` du package. La NavBar bascule, « Nouvel evenement » apparaît — la gestion d'auth partagée fonctionne comme avant.

FIGURE 3 `/evenement/:id`

Quatre modules partagés dans un seul écran



« Organisé par : Atou Diop » vient de la jointure de `getEvenements()`. Le bouton « Se désinscrire » prouve que `estInscrit()` a retourné vrai. « Supprimer » n'apparaît que pour l'organisateur, prêt à appeler `supprimerEvenement()`. Événements, auth, inscriptions : toute la logique passe désormais par `@senevent/shared`, sans une seule régression.

Vers le Lab 8

Le socle partagé est en place : la logique métier est écrite une fois, typée, et validée par le web. Au Lab 8, on démarre le mobile avec Expo et React Native — `apps/mobile/` à côté de `apps/web/`. On découvre les composants natifs (`View`, `Text`, `StyleSheet`) qui remplacent `<div>` et le CSS. Et surtout : `apps/mobile` importera `@senevent/shared` exactement comme le web. Le premier écran mobile qui affiche les événements réutilisera `getEvenements()` sans une ligne de logique Supabase à réécrire. C'est là que le travail de ce lab paiera.